

Azure Synapse Analytics

Synapse SQL, Spark, pipelines, distributed architecture, performance, and trade-offs

Curated study notes for the RAG learning project. Synthesized from official documentation; verify current product behavior in official docs before production use.

1. Azure Synapse Analytics Overview

Azure Synapse Analytics is an enterprise analytics service that brings together SQL technologies, Apache Spark, data integration, data lake access, and related analytics capabilities.

Key idea: Synapse is a unified analytics workspace, not just a SQL database. Its components target different analytical workloads.

Core capabilities

Synapse includes Synapse SQL, Apache Spark, pipelines/data integration, and Data Explorer capabilities. It integrates with Azure storage and analytics services.

- SQL supports warehouse and data-virtualization scenarios.
- Spark supports big-data engineering and data science.
- Pipelines orchestrate integration.
- Storage commonly uses Azure Data Lake Storage.

When to use

Synapse can fit organizations that want a unified analytics environment or need distributed SQL, Spark, and orchestration capabilities in an Azure-native service.

- Evaluate workload requirements before choosing dedicated or serverless SQL.
- Consider Databricks when Spark/lakehouse engineering is the dominant requirement.

2. Synapse SQL Architecture

Synapse SQL uses distributed query processing. A control node receives queries and coordinates execution across compute resources, while data resides in Azure Storage.

Key idea: Synapse SQL separates compute from storage, allowing compute resources to scale independently of the underlying data.

Dedicated SQL pool

Dedicated SQL pool provides provisioned compute for data warehousing. Capacity is measured using data warehouse units (DWUs).

- Suitable for predictable, high-volume analytical workloads.
- Performance depends on distribution, data movement, statistics, and query design.

Serverless SQL pool

Serverless SQL queries data in external storage without requiring provisioned dedicated compute. Scaling is managed by the service.

- Useful for ad hoc exploration and querying lake data.
- Cost is typically related to data processed, so avoid unnecessary scans.
- Use file formats and partitioning that support efficient pruning.

Control and compute nodes

The control node is the entry point and optimizer/coordinator. Compute nodes execute work in parallel.

- Data movement between nodes can be expensive.
- Distribution choices affect parallelism and data movement.

3. Distributed Tables, Distribution, and Performance

Dedicated Synapse SQL distributes table data across compute nodes. Distribution design can strongly influence join and aggregation performance.

Key idea: Choose distribution based on common joins and data volume, while avoiding severe skew.

Distribution methods

Common strategies include hash distribution, round-robin distribution, and replicated tables. Exact capabilities depend on the Synapse SQL mode.

- Hash distribution places rows using a distribution key.
- Round-robin spreads rows without key-based locality.
- Replication keeps small dimension-like tables on each node.

Data movement

When data required for a query is not colocated, Synapse may move data between compute nodes. This can become a major cost.

- Choose hash keys that match common large-table joins.
- Replicate small tables when appropriate.
- Inspect query plans and data movement operations.

Statistics

Statistics help the optimizer estimate row counts and choose execution strategies.

- Keep statistics current.
- Skewed distributions can invalidate assumptions.

4. Synapse Spark

Synapse provides serverless Apache Spark pools for data engineering, notebooks, and analytics. Spark workloads operate over data in Azure Storage and other supported sources.

Key idea: Synapse Spark uses the Spark execution model: driver, executors, partitions, stages, tasks, and shuffles.

Spark pools

Spark pools provide managed compute for Spark sessions. They can scale resources and can shut down without losing data stored in external storage.

- Choose node size based on workload.
- Use autoscale where appropriate.
- Avoid storing durable data only on ephemeral compute.

Notebooks

Synapse notebooks support PySpark, Scala, Spark SQL, and other supported languages. They are useful for interactive engineering and data exploration.

- Keep notebooks modular.
- Parameterize production workloads.
- Version production code.

Integration with lake

Spark can read/write files and tables in Azure Data Lake Storage. Synapse pipelines can orchestrate Spark notebooks and jobs.

- Separate storage from compute.
- Use efficient formats such as Parquet/Delta where supported by the architecture.

5. Synapse Pipelines and Integration

Synapse pipelines provide data integration and orchestration capabilities closely related to Azure Data Factory.

Key idea: The ADF mental model transfers well: pipeline, activity, dataset, linked service, integration runtime, trigger, parameters, and monitoring.

Pipelines

A pipeline groups activities and defines dependencies, parameters, and execution logic.

- Use parallelism where dependencies allow.
- Use child pipelines for reuse.
- Design failure paths.

Activities and connections

Activities perform copy, transformation, notebook, SQL, and control operations. Linked services define connections; integration runtime provides the execution bridge.

- Use managed identities where supported.
- Keep secrets out of code.

Operational design

Monitor runs, duration, failures, and data volumes. Alerts should focus on meaningful SLA breaches.

- Test reruns.
- Document source and target ownership.
- Make pipelines parameter-driven.

6. Synapse Reference Architecture and Trade-offs

Synapse can support a lake-centric architecture in which ADLS stores data, Spark performs engineering, Synapse SQL serves analytics, and pipelines orchestrate movement and transformation.

Key idea: Synapse and Azure Databricks overlap in Spark-based engineering. Choose based on team skills, governance, workload, integration, and platform strategy.

Reference flow

ADLS can act as the durable storage layer. Synapse Spark transforms data; Synapse SQL provides analytical serving; pipelines coordinate workflows.

- Sources → pipelines → ADLS.
- ADLS → Spark → curated data.
- Curated data → SQL pool/BI.
- Monitoring and governance span the system.

Synapse vs Databricks

Both can run Spark and support data engineering. Databricks is strongly centered on the lakehouse and Spark ecosystem; Synapse emphasizes an integrated Azure analytics workspace combining SQL, Spark, and pipelines.

- Use workload requirements rather than product popularity.
- Consider existing Azure investments and operational standards.
- Avoid duplicate platforms without a clear purpose.

Performance checklist

For SQL, examine distribution, data movement, statistics, file layout, and query design. For Spark, use partitioning, joins, shuffle analysis, and Spark UI.

- Measure cost as well as runtime.
- Use representative data volumes.
- Test concurrency when serving BI workloads.

Selected official sources

- <https://learn.microsoft.com/en-in/azure/synapse-analytics/overview-what-is>
- <https://learn.microsoft.com/en-us/azure/synapse-analytics/sql/overview-architecture>
- <https://learn.microsoft.com/en-us/azure/synapse-analytics/overview-terminology>
- <https://learn.microsoft.com/en-us/azure/synapse-analytics/spark/apache-spark-overview>